

UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA



Facultad de Computación - Escuela de Postgrado
Maestría en Ciencia de Datos e IA

Proyecto Final – Entrega Parcial
Análisis de Performance de un Solver MPI
para la Ecuación de Calor en 2D

Grupo N°1

Alumnos:

Ángeles Chavez, Hugo
Cajusol Mio, Christian Edgardo
Meza Medel, Francisco Ernesto
Yurivilca Gomez, Jhomar David

Docente: Dr. Jose Antonio, Fiestas Iquira

Applied High Performance Computing - AHPC
Ciclo - IV

NO PUBLICABLE

2026

BARRANCO-PERÚ

Índice

1. Datos del Proyecto y Participación del Equipo	3
2. Introducción	3
3. Descripción del Problema y del Algoritmo	4
3.1. La ecuación de calor	4
3.2. Discretización: diferencias finitas y stencil de 5 puntos	4
3.3. Esquema explícito (Jacobi) y estabilidad CFL	5
3.4. El problema concreto y la configuración experimental	5
3.5. Algoritmo paralelo: descomposición de dominio MPI y halos	5
4. Método: Modelo PRAM y Complejidad Teórica	7
4.1. El modelo PRAM y por qué aplica aquí	7
4.2. El algoritmo como programa PRAM	7
4.3. DAG y análisis de costos por iteración	8
4.4. Número de iteraciones hasta converger: $T = \Theta(n^2)$	10
4.5. Trabajo total, profundidad y paralelismo disponible	10
4.6. Tiempo, <i>speedup</i> y eficiencia en el modelo PRAM	11
4.7. Del PRAM a la realidad: memoria distribuida y halos (MPI)	11
4.8. Número óptimo de procesos y escalabilidad débil	12
4.9. Resumen de complejidades	13
5. Metodología Experimental y Métricas	14
5.1. Definición de las métricas	14
6. Bibliografía y Uso de IA	14
6.1. Referencias	14
6.2. Uso de Inteligencia Artificial	15

1. Datos del Proyecto y Participación del Equipo

Datos generales

Campo	Valor
Curso	CS8084 – Applied High Performance Computing (AHPC)
Ciclo	IV – Maestría en Ciencia de Datos e IA, Posgrado UTEC 2026-I
Docente	Dr. Jose Antonio Fiestas Iquira
Tema elegido	Opción 4 del enunciado: <i>Ecuación de Calor</i>
Repositorio	https://github.com/hugoangeles0810/ahpc-heat-equation
Clúster de cómputo	Khipu (UTEC), cuenta postgrado , nodos Xeon Gold 6130
Entrega	Entrega Parcial (sección 7.1 del enunciado del proyecto)

Integrantes y porcentaje de participación

La Tabla 1 resume el porcentaje de participación de cada integrante en el proyecto. El reparto de trabajo combina desarrollo de código/experimentación en el clúster, redacción del marco teórico (PRAM y complejidad) y consolidación del informe.

Integrante	% participación
Hugo Ángeles Chávez	100 %
Christian Cajusol Mio	100 %
Francisco Meza Medel	100 %
Jhomar Yurivilca Gómez	100 %

Cuadro 1: Participación del equipo en la Entrega Parcial.

2. Introducción

El cómputo de alto desempeño (*High Performance Computing*, HPC) tiene en la solución numérica de ecuaciones diferenciales parciales (EDP) uno de sus casos de uso más representativos: simulaciones de clima, mecánica de fluidos, electromagnetismo y transferencia de calor se reducen, en la práctica, a resolver mallas con millones de incógnitas mediante esquemas iterativos que se repiten miles de veces hasta alcanzar convergencia. Este tipo de algoritmos son **intensivos en cómputo y en comunicación**, y son el escenario natural para aplicar el modelo de paralelismo PRAM y el modelo de costos de comunicación, junto con las herramientas (MPI, SLURM) estudiadas en el curso.

De las opciones propuestas en el enunciado del proyecto, el grupo eligió la **Opción 4 – Ecuación de Calor**: analizar el desempeño de un programa que resuelve la ecuación de calor 2D en paralelo mediante el método de diferencias finitas con un esquema explícito (Jacobi). El proyecto contempla:

1. Una versión **serial** de referencia (`src/heat-big.c`), con el mismo método de Jacobi pero sin MPI. Esta versión se incluye solo como referencia del algoritmo: **no** es la que se usa para calcular el *speedup*. Para eso se usa la propia versión **MPI ejecutada con $p = 1$ proceso** como caso base T_1 , de modo que $S(p) = T(n, 1)/T(n, p)$ siempre compara el mismo programa, cambiando solo el número de procesos.

2. Una versión **paralela con MPI** (`src/heat-mpi-big.c`) que descompone el dominio 2D entre p procesos mediante un comunicador cartesiano, intercambia *halos* (celdas fantasma) en cada iteración y evalúa el criterio de convergencia global mediante `MPI_Allreduce`.
3. Infraestructura de **benchmarking** sobre el clúster Khipu (UTEC) mediante SLURM, que ejecuta un barrido de tamaños de malla $n \in \{128, 256, 512, 1024\}$ y número de procesos $p \in \{1, 2, 4, 8, 16\}$.
4. Un **análisis de performance** (`analysis/plot_performance.py`) que procesa los resultados y genera las gráficas de tiempo, *speedup*, eficiencia e iteraciones hasta convergencia.

El proyecto contempla tres objetivos:

- (a) Elaborar el modelo **PRAM** del algoritmo, determinar su complejidad teórica y definir las métricas de *speedup* y eficiencia (Secciones 4 y 5.1).
- (b) Ejecutar mediciones de tiempo para distinto número de procesos y tamaño de problema, y compararlas con la complejidad teórica.
- (c) Desarrollar un software de análisis de performance que genere la representación gráfica de las métricas y permita concluir sobre la escalabilidad.

Este **informe parcial** cubre el objetivo (a) – modelo PRAM y definición de métricas – junto con la descripción del problema/algoritmo y la bibliografía utilizada. La infraestructura de *benchmarking* en Khipu (objetivo b) y el software de análisis (objetivo c) ya están implementados en el repositorio; sus resultados y la discusión correspondiente se presentarán en el informe final.

El resto del documento se organiza así: la Sección 3 describe el problema físico, su discretización y el algoritmo paralelo; la Sección 4 desarrolla el modelo PRAM, el análisis trabajo–profundidad y su extensión a memoria distribuida; la Sección 5 define las métricas de *speedup* y eficiencia que se evaluarán con datos reales; y la Sección 6 documenta las fuentes externas utilizadas (incluyendo IA).

3. Descripción del Problema y del Algoritmo

3.1. La ecuación de calor

La ecuación de calor describe cómo se redistribuye la temperatura $u(x, y, t)$ en un dominio a lo largo del tiempo [1]. En dos dimensiones, con difusividad $\alpha = 1$:

$$\frac{\partial u}{\partial t} = \alpha \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) = \alpha \nabla^2 u.$$

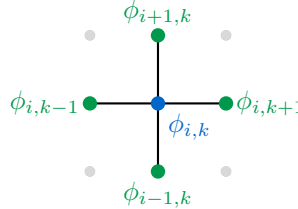
Intuitivamente, el calor fluye de lo caliente a lo frío y la rapidez de cambio de un punto depende de cuánto difiere de sus vecinos (curvatura, $\nabla^2 u$). Cuando el sistema deja de cambiar ($\partial u / \partial t = 0$), se llega al **estado estacionario**, que satisface la ecuación de Laplace $\nabla^2 u = 0$. El programa resuelve precisamente esto: itera en el tiempo hasta que la temperatura deja de cambiar (criterio $|\Delta \phi|_{\text{máx}} < \text{eps}$), obteniendo así la distribución de equilibrio.

3.2. Discretización: diferencias finitas y stencil de 5 puntos

El dominio continuo se reemplaza por una malla de $n \times n$ nodos interiores separados por $dx = dy = 1/n$. Las segundas derivadas se aproximan con diferencias centradas [2]:

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{\phi_{i,k+1} - 2\phi_{i,k} + \phi_{i,k-1}}{dx^2}, \quad \frac{\partial^2 u}{\partial y^2} \approx \frac{\phi_{i+1,k} - 2\phi_{i,k} + \phi_{i-1,k}}{dy^2}.$$

Sumando ambas, cada nodo se actualiza usando **a sí mismo y a sus 4 vecinos directos** – el clásico *stencil* de 5 puntos:



3.3. Esquema explícito (Jacobi) y estabilidad CFL

El esquema explícito (*Forward-Time Centered-Space*, FTCS) [2] actualiza cada nodo a partir de valores del paso anterior:

$$\phi_{i,k}^{n+1} = \phi_{i,k}^n + dt \left[\frac{\phi_{i+1,k}^n + \phi_{i-1,k}^n - 2\phi_{i,k}^n}{dy^2} + \frac{\phi_{i,k+1}^n + \phi_{i,k-1}^n - 2\phi_{i,k}^n}{dx^2} \right].$$

Para que el esquema sea **estable** (análisis de von Neumann), el paso de tiempo debe satisfacer la condición CFL [3, 7]:

$$dt \leq \frac{1}{2} \frac{dx^2 dy^2}{dx^2 + dy^2} \xrightarrow{dx=dy} dt = \frac{\min(dx^2, dy^2)}{4} = \Theta(1/n^2),$$

que es exactamente lo que implementa el código (`dt = min(dx2,dy2)/4.0`). Esta dependencia $dt = \Theta(1/n^2)$ es la raíz de la complejidad teórica derivada en la Sección 4: a mayor resolución de malla, más pasos de tiempo se necesitan para llegar al estado estacionario.

3.4. El problema concreto y la configuración experimental

El dominio es el cuadrado unitario $[0, 1] \times [0, 1]$ con condiciones de frontera tipo **Dirichlet**: borde derecho $\phi = 1$ (caliente), borde izquierdo $\phi = 0$ (frío), bordes superior e inferior con una rampa lineal $\phi = k \cdot dx$, e interior inicializado en 0. Partiendo de un interior frío, el calor difunde desde los bordes hasta el equilibrio (gradiente diagonal frío→caliente).

Para el barrido de desempeño (objetivo b) se usan mallas $n \in \{128, 256, 512, 1024\}$ (parámetro `grid_size` del binario) con un tope de `itmax = 20000` iteraciones y tolerancia `eps = 1e-8`, ejecutadas con $p \in \{1, 2, 4, 8, 16\}$ procesos MPI sobre el clúster Khipu.

3.5. Algoritmo paralelo: descomposición de dominio MPI y halos

La versión MPI (`heat-mpi-big.c`) reparte la malla en una rejilla cartesiana $idim \times kdim$ tan cuadrada como sea posible (`MPI_Dims_create + MPI_Cart_create`), de modo que cada proceso posee un subdominio de $\approx (n/\sqrt{p}) \times (n/\sqrt{p})$ nodos.

Como cada proceso solo guarda su **subdominio** en memoria, el *stencil* en una celda del borde necesita valores de `phi` de un proceso VECINO – algo que PRAM (memoria compartida) no contempla.

La solución: cada proceso añade una fila/columna EXTRA alrededor de su subdominio, los **halos** (*ghost cells*), con una copia local del borde vecino. Al final de cada iteración se INTERCAMBIAN esas filas/columnas (Código 1, líneas 423–427) usando tipos derivados (`MPI_Type_create_subarray`) y comunicación no bloqueante [4].

El Código 1 muestra solo el **cuerpo del bucle** (líneas 239–297 de 350): la inicialización corre *una vez* y queda dominada por las $T = \Theta(n^2)$ repeticiones (Sección 4.4). Implementa exactamente las tres fases que se formalizan como PRAM en la Sección 4:

- (1) cómputo del *stencil* y copia Jacobi
- (2) reducción global del criterio de convergencia cada `stride=10` pasos
- (3) intercambio de halos.

Código 1: Núcleo del bucle de iteración (`src/heat-mpi-big.c`, líneas 239–297, abreviado).

```
1 for (it = 1; it <= itmax; it++) {
2     double dphimax = 0.;
3     for (i = is + halo; i <= ie - halo; i++) {
4         for (k = ks + halo; k <= ke - halo; k++) {
5             double dphi = (phi[idx(i+1,k)] + phi[idx(i-1,k)] - 2.*phi[idx(i,k)]) * dy2i
6                           + (phi[idx(i,k+1)] + phi[idx(i,k-1)] - 2.*phi[idx(i,k)]) * dx2i;
7             dphi *= dt;
8             dphimax = dmax(dphimax, dphi);
9             phin[idxn(i,k)] = phi[idx(i,k)] + dphi;           // Jacobi: doble buffer
10        }
11    }
12    for (i = is+halo; i <= ie-halo; i++)
13        for (k = ks+halo; k <= ke-halo; k++)
14            phi[idx(i,k)] = phin[idxn(i,k)];                  // copia phin -> phi
15
16    if ((it % stride) == 0) {                                // criterio cada "stride" pasos
17        if (numprocs > 1)
18            MPI_Allreduce(&dphimax, &dphimax, 1, MPI_DOUBLE, MPI_MAX, comm);
19        if (dphimax < eps) break;                            // estado estacionario
20    }
21
22    // intercambio de halos (no bloqueante) con los 4 vecinos cartesianos
23    MPI_Irecv(...); MPI_Irecv(...); MPI_Isend(...); MPI_Isend(...);
24    MPI_Waitall(4, req, statuses);                           // arriba/abajo
25    MPI_Irecv(...); MPI_Irecv(...); MPI_Isend(...); MPI_Isend(...);
26    MPI_Waitall(4, req, statuses);                           // izquierda/derecha
27 }
```

El programa imprime, además, una línea con el siguiente formato, que es el insumo directo del software de análisis (Sección 5):

```
RESULT,grid_size,nprocs,idim,kdim,
iterations,wall_time_s,comm_time_s,criterion_time_s
```

4. Método: Modelo PRAM y Complejidad Teórica

Esta sección desarrolla el **objetivo (a)** y el punto 1 de la rúbrica (“*el diseño del algoritmo es adecuado y las métricas corresponden al PRAM*”). La notación usada en esta sección es:

- n = lado de la malla interior ($N = n^2$ nodos)
- p = número de procesadores
- T = número de iteraciones hasta converger
- W_1 = trabajo total
- D_∞ = profundidad

4.1. El modelo PRAM y por qué aplica aquí

El modelo **PRAM** (*Parallel Random Access Machine*) asume p procesadores síncronos que comparten memoria global y, en cada paso, ejecutan una operación $O(1)$ (leer/calcular/escribir) [5]. Es el análogo paralelo del modelo RAM secuencial e ignora deliberadamente el costo de comunicación, para aislar el **paralelismo intrínseco** del algoritmo. Sus variantes se distinguen por cómo resuelven accesos simultáneos a una misma celda:

Variante	Lectura concurrente	Escritura concurrente
EREW	✗	✗
CREW	✓	✗
CRCW	✓	✓

El solver es naturalmente CREW:

- **Lectura concurrente (CR):** el *stencil* de 5 puntos hace que cada celda $\phi_{i,k}$ sea **leída por hasta 5 procesadores** a la vez (el dueño de la celda y sus 4 vecinos), siempre sobre el arreglo *viejo* ϕ .
- **Escritura exclusiva (EW):** el esquema de **Jacobi con doble buffer** escribe cada resultado en un arreglo separado ϕ_{in} – una celda por procesador, sin colisiones – y solo después se copia $\phi_{\text{in}} \rightarrow \phi$.

Cada punto depende solo de sus vecinos inmediatos y los nuevos valores no se pisan entre sí: esto hace que el problema sea **vergonzosamente paralelo dentro de cada paso de tiempo**, y es la base de toda la escalabilidad analizada en esta sección. El modelo PRAM da la cota *optimista* (sin comunicación); la Sección 4.7 añade el costo real de halos y reducción.

4.2. El algoritmo como programa PRAM

El bucle de `heat-mpi-big.c` (Código 1) se reescribe en notación PRAM en el Código 2. `forall` denota un bucle cuyas iteraciones se reparten entre los p procesadores y se ejecutan en paralelo.

¿Por qué el Código 2 no incluye los halos? Aunque su intercambio (Sección 3.5) también se repite T veces, PRAM es de memoria compartida y no tiene primitivo de mensajes – se modela aparte, agregando el costo de comunicación, en la Sección 4.7.

Código 2: Algoritmo PRAM de una iteración del solver.

```

1 Entrada: malla phi[0..n+1,0..n+1] con condiciones de frontera (Dirichlet)
2 Parametros: dt = min(dx^2,dy^2)/4      (limite de estabilidad CFL)
3             eps = 1e-8                  (tolerancia de convergencia)
4
5 para it = 1,2,3,...                      // SECUENCIAL: cada paso usa el anterior
6   (1) forall (i,k) en el interior [n^2 tareas independientes] // 0(1) c/u
7       dphi      <- (phi[i+1,k]+phi[i-1,k]-2*phi[i,k]) * dy2i
8               + (phi[i,k+1]+phi[i,k-1]-2*phi[i,k]) * dx2i
9       dphi      <- dphi * dt
10      phin[i,k]  <- phi[i,k] + dphi
11      local[i,k] <- |dphi|                // contribucion al criterio de parada
12
13   (2) forall (i,k) en el interior        // copia Jacobi  phi <- phin  [0(1) c/u]
14       phi[i,k] <- phin[i,k]
15
16   (3) dphimax <- REDUCE_MAX( local[i,k] ) // reduccion en arbol: 0(log n) de
17       profundidad
18       si dphimax < eps -> terminar      // estado estacionario alcanzado

```

Tres observaciones clave para el análisis:

1. **Dentro de un paso**, los bloques (1) y (2) son **forall** puros. La anotación “ n^2 tareas independientes de costo $O(1)$ ” del Código 2 se justifica en tres pasos:
 - n^2 *tareas*: el doble bucle **for**(i) **for**(k) del Código 1 recorre las n filas y n columnas *interiores* de la malla; cada par (i, k) es una iteración $\Rightarrow n \times n = n^2$ tareas.
 - *Costo* $O(1)$: el cuerpo de la tarea (bloque (1) del Código 2) calcula **dphi** a partir de los 4 vecinos, lo escala por **dt** y escribe **phin[i,k]** y **local[i,k]** – siempre el mismo número fijo de operaciones, sin bucles ni recursión, sin depender de n , p ni (i, k) .
 - *Independencia*: cada tarea (i, k) solo **lee phi** (que nadie modifica en este paso) y solo **escribe** su propia celda **phin[i,k]**: ninguna otra tarea toca esa celda, así que no hay dependencias de datos entre ellas.

Por la regla del **forall** puro, m tareas independientes de costo $O(1)$ aportan trabajo $O(m)$ y, con $p \geq m$ procesadores, profundidad $O(1)$. Con $m = n^2$: trabajo $O(n^2)$ y profundidad $O(1)$ (si $p \geq n^2$).

2. **El criterio de parada (3)** no es independiente: combinar el máximo de n^2 valores es una **reducción**, cuya profundidad óptima es $O(\log n^2) = O(\log n)$ (árbol binario) y trabajo $O(n^2)$.
3. **Entre pasos** existe una dependencia estricta: ϕ^{n+1} necesita ϕ^n completo. El bucle **para it** es **inherentemente secuencial**; el paralelismo vive *dentro* de cada paso, fijando el límite inferior de profundidad del algoritmo completo.

4.3. DAG y análisis de costos por iteración

La Figura 1 muestra el grafo de dependencias (DAG) de una iteración temporal, que se repite T veces de forma secuencial. Las fórmulas de esta sección y las siguientes se obtienen sumando o multiplicando estos costos por T (Sección 4.4), o aplicando la reducción en árbol y el modelo de costos de comunicación, derivando $T_{par}(n, p)$ respecto a p para obtener el número óptimo de procesos: ningún paso queda sin justificar – la Tabla 4, al cierre de la Sección 4, recapitula de dónde sale cada fórmula nueva.

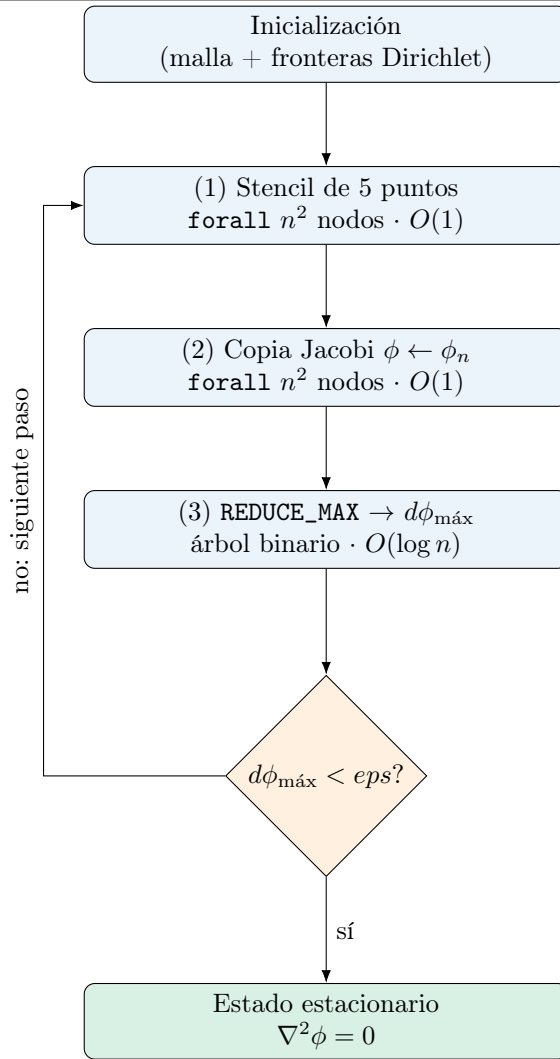


Figura 1: DAG de una iteración temporal. El bloque punteado se repite $T = \Theta(n^2)$ veces; el ciclo completo es la cadena de dependencias secuencial.

La Tabla 2 anota el costo de cada fase sobre el modelo PRAM con p procesadores.

Fase	Trabajo	Profundidad	Tiempo con p proc.
(1) Stencil	$\Theta(n^2)$	$\Theta(1)$	$\Theta(n^2/p)$
(2) Copia Jacobi	$\Theta(n^2)$	$\Theta(1)$	$\Theta(n^2/p)$
(3) Reduce máximo	$\Theta(n^2)$	$\Theta(\log n)$	$\Theta(n^2/p + \log p)$
Por iteración	$\Theta(n^2)$	$\Theta(\log n)$	$\Theta(n^2/p + \log p)$

Cuadro 2: Costo PRAM por fase y por iteración (sin comunicación).

El cuello de botella de **profundidad** dentro de un paso es la reducción del criterio de parada, $\Theta(\log n)$; todo lo demás es $\Theta(1)$. En el código MPI esa reducción es el `MPI_Allreduce`, que por eso se ejecuta solo cada `stride=10` pasos (Código 1) – una optimización que el modelo justifica directamente.

4.4. Número de iteraciones hasta converger: $T = \Theta(n^2)$

La complejidad total depende de cuántos pasos T se necesitan, y T **no es constante**: crece con la malla, como consecuencia directa del esquema explícito.

Estabilidad (CFL). Como se vio en la Sección 3, la condición de estabilidad de von Neumann fija $dt = \Theta(1/n^2)$ [6, 7].

Tasa de convergencia. Iterar el esquema explícito hasta el estado estacionario equivale a una **iteración de Jacobi** sobre la ecuación de Laplace. El error se descompone en modos de Fourier; el modo más lento decae por iteración con un factor $\approx 1 - \Theta(dt \cdot \lambda_{\min})$, donde $\lambda_{\min} = 2\pi^2$ es el menor autovalor del Laplaciano en el cuadrado unitario (constante, independiente de n). El número de pasos para llevar ese modo bajo ϵ es

$$T = \Theta\left(\frac{1}{dt \lambda_{\min}}\right) = \Theta\left(\frac{1}{dt}\right) = \Theta(n^2).$$

Validación empírica preliminar. En una malla 80×80 ($n^2 = 6400$) el solver converge en $\approx 14,300$ iteraciones (ver docs/heat-equation.md). El cociente $14300/6400 \approx 2,2$ es la constante del $\Theta(\cdot)$, y debería mantenerse aproximadamente constante al variar n – una predicción concreta y verificable con el objetivo (b).

Implicación HPC: afinar la malla cuesta doble. Duplicar n multiplica por 4 el trabajo por paso y por 4 el número de pasos $\Rightarrow$ el costo serial crece como n^4 . De ahí la necesidad de paralelizar.

4.5. Trabajo total, profundidad y paralelismo disponible

Combinando el costo por iteración (Tabla 2) con $T = \Theta(n^2)$ (Sección 4.4):

Trabajo total (= tiempo secuencial, 1 procesador):

$$W_1 = T_1 = \Theta(T \cdot n^2) = \Theta(n^2 \cdot n^2) = \Theta(n^4) = \Theta(N^2)$$

Profundidad total (cadena crítica): los T pasos son secuenciales y cada uno aporta profundidad $\Theta(\log n)$:

$$D_\infty = \Theta(T \cdot \log n) = \Theta(n^2 \log n)$$

Paralelismo disponible (procesadores que el algoritmo puede aprovechar):

$$\frac{W_1}{D_\infty} = \frac{\Theta(n^4)}{\Theta(n^2 \log n)} = \Theta\left(\frac{n^2}{\log n}\right).$$

Es decir, el algoritmo puede usar útilmente hasta $\approx n^2$ procesadores (uno por nodo de la malla, salvo el factor $\log n$ de la reducción); más allá de eso, los procesadores extra quedan ociosos. La Sección 4.6 reparte directamente W_1 entre p procesadores (sumando el costo de la reducción) para obtener T_p .

4.6. Tiempo, *speedup* y eficiencia en el modelo PRAM

A partir del costo por iteración $\Theta(n^2/p + \log p)$ (Tabla 2) y $T = \Theta(n^2)$:

$$T_p = \Theta\left(n^2\left(\frac{n^2}{p} + \log p\right)\right) = \Theta\left(\frac{n^4}{p} + n^2 \log p\right)$$

Con el máximo paralelismo $p = n^2$: $T_{n^2} = \Theta(n^2 \log n)$, que coincide con D_∞ – el algoritmo es *óptimo en profundidad*.

Speedup (respecto del óptimo secuencial $T_1 = \Theta(n^4)$):

$$S_p = \frac{T_1}{T_p} = \frac{n^4}{n^4/p + n^2 \log p} = \frac{p}{1 + \frac{p \log p}{n^2}}.$$

Eficiencia:

$$E_p = \frac{S_p}{p} = \frac{1}{1 + \frac{p \log p}{n^2}}$$

- $E_p \rightarrow 1$ (*speedup* casi lineal) **mientras** $n^2 \gg p \log p$.
- La eficiencia se degrada cuando p se acerca a n^2 : justo cuando el trabajo por procesador (n^2/p) se vuelve comparable al costo $\log p$ de la reducción.

Costo (procesadores $\times$ tiempo) y *cost-óptimalidad*:

$$C_p = p \cdot T_p = \Theta(n^4 + p n^2 \log p).$$

El algoritmo paralelo es **cost-óptimal** (mismo orden que el trabajo secuencial $\Theta(n^4)$) siempre que $p \log p = O(n^2)$, es decir

$$p = O\left(\frac{n^2}{\log n}\right),$$

la cota teórica del número útil de procesos en el modelo idealizado. La Sección 4.7 la corrige a la baja al incluir comunicación real.

4.7. Del PRAM a la realidad: memoria distribuida y halos (MPI)

El modelo PRAM supone comunicación gratuita. El código real es de **memoria distribuida**: cada proceso solo ve su porción de la malla y debe **comunicar** explícitamente. Modelamos el costo de comunicación con el modelo de latencia–ancho de banda visto en el curso (Unidad 2): enviar un mensaje de m palabras cuesta $t_s + t_w m$, donde t_s es la latencia (*startup*) y t_w el costo por palabra (inverso del ancho de banda).

Descomposición de dominio 2D. MPI_Dims_create reparte la malla en una rejilla lo más cuadrada posible, $\sqrt{p} \times \sqrt{p}$ – el caso $d = 2$ de una malla d -dimensional con p nodos (Unidad 4.3, “Topologías”),

donde el lado de cada subdominio escala como $p^{1/d}$. Cada proceso posee un subdominio de $\frac{n}{\sqrt{p}} \times \frac{n}{\sqrt{p}} =$

$\frac{n^2}{p}$ nodos (Sección 3.5).

La Tabla 3 descompone el costo **por iteración** de un proceso.

Componente	Costo	Origen en el código
Cómputo	$\Theta\left(\frac{n^2}{p}\right)$	doble bucle del <i>stencil</i> (Código 1)
Halos	$\Theta\left(t_s + t_w \frac{n}{\sqrt{p}}\right)$	Irecv/Isend/Waitall, 4 vecinos
Criterio	$\Theta\left(\frac{t_s \log p}{\text{stride}}\right)$	MPI_Allreduce cada stride pasos

Cuadro 3: Costo por iteración de un proceso (cómputo + comunicación).

La clave es el **cociente superficie/volumen** comunicación-a-cómputo por paso:

$$\frac{\text{comm}}{\text{comp}} = \frac{n/\sqrt{p}}{n^2/p} = \frac{\sqrt{p}}{n}.$$

El cómputo crece con el **área** del subdominio (n^2/p); la comunicación solo con su **perímetro** ($n/\sqrt{p}$). Por eso una malla grande amortiza mejor la comunicación: la eficiencia se mantiene mientras $n \gg \sqrt{p}$. **Tiempo paralelo total** (sobre $T = \Theta(n^2)$ pasos):

$$T_{\text{par}}(n, p) = \underbrace{\Theta\left(\frac{n^4}{p}\right)}_{\text{cómputo}} + \underbrace{\Theta\left(\frac{n^3}{\sqrt{p}}\right)}_{\text{halos (ancho de banda)}} + \underbrace{\Theta\left(\frac{n^2 t_s \log p}{\text{stride}}\right)}_{\text{reducción (latencia)}}.$$

Eficiencia con comunicación:

$$E(n, p) = \frac{1}{1 + \Theta\left(\frac{t_w \sqrt{p}}{n}\right) + \Theta\left(\frac{t_s p \log p}{n^2}\right)}.$$

El término dominante al crecer p es el de ancho de banda $\Theta(\sqrt{p}/n)$ – la huella de superficie/volumen. Esto explica cualitativamente lo que se observa en el objetivo (b): para n pequeño la eficiencia cae rápido al aumentar p (el código serial puede incluso ganar por el *overhead*), mientras que para n grande el *speedup* se acerca al lineal – el mismo fenómeno documentado en [9, 10].

4.8. Número óptimo de procesos y escalabilidad débil

Número óptimo de procesos (para n fijo). Igual que en el ejemplo de KNN visto en clase (Unidades 4.2/4.3) – donde el punto óptimo de procesos se obtiene derivando $T(n, p)$ respecto a p e igualando a cero –, minimizamos $T_{\text{par}}(n, p)$ en p : el término de cómputo $\propto 1/p$ decrece y el de comunicación $\propto \sqrt{p}$ crece, así que $\partial T_{\text{par}}/\partial p = 0$ se cumple donde el cómputo por proceso iguala a la comunicación, $c_f n^2/p \approx t_w n/\sqrt{p}$:

$$p_{\text{opt}} = \Theta\left(\left(\frac{c_f}{t_w}\right)^2 n^2\right) = \Theta(n^2).$$



Coincide en orden con la cota PRAM ($p = O(n^2/\log n)$, Sección 4.6): no tiene sentido usar más de $\approx n^2$ procesos. En la práctica la constante $(c_f/t_w)^2$ es pequeña, así que el óptimo medido estará bastante por debajo de n^2 ; el barrido $p \times n$ del objetivo (b) sirve precisamente para calibrar esa constante.

Escalabilidad débil. ¿Cómo debe crecer n con p para mantener $E(n, p)$ (Sección 4.7) aproximadamente constante? El término que domina al crecer p es el de ancho de banda, $\Theta(t_w\sqrt{p}/n)$; mantenerlo constante exige

$$n = \Theta(\sqrt{p}) \iff W_1 = n^4 = \Theta(p^2).$$

Es una escalabilidad **moderada**: para conservar la eficiencia hay que aumentar el problema aproximadamente con el cuadrado del número de procesos – típico de *stencils* de diferencias finitas.

Resumen de complejidades ¿de dónde sale cada fórmula? Antes de consolidar los valores finales (Tabla 5), la Tabla 4 recorre las subsecciones 4.3–4.8 y resume, para cada una, qué resultado nuevo agrega y de dónde proviene: conteo directo, resultado citado, o una multiplicación/sustitución algebraica sobre lo anterior – ningún paso queda sin justificar.

Subsec.	Qué resultado nuevo agrega y dónde se origina
4.2	$W = O(n^2)$, $D = O(1)$ para (1)/(2) $\leftarrow$ conteo directo: n^2 tareas de costo $O(1)$ (justificado arriba)
4.3	REDUCE_MAX : $\Theta(n^2)$ trabajo, $\Theta(\log n)$ profundidad $\leftarrow$ reducción en árbol binario (resultado estándar); el total “por iteración” es la suma de las 3 fases
4.4	$T = \Theta(n^2) \leftarrow$ no viene de PRAM: condición CFL ($dt = \Theta(1/n^2)$) más tasa de convergencia de Jacobi (dato externo, Sección 4.4)
4.5	$W_1 = \Theta(n^4)$, $D_\infty = \Theta(n^2 \log n)$, paralelismo disponible $W_1/D_\infty = \Theta(n^2/\log n) \leftarrow T \times (\text{costo/iteración, Tabla 2})$; cociente directo (álgebra)
4.6	$T_p, S_p, E_p \leftarrow T_p = T \times \text{tiempo/iter. con } p$; $S_p = T_1/T_p$, $E_p = S_p/p$ (definiciones, álgebra)
4.7	Costo halo/reducción $\leftarrow$ cómputo n^2/p , halo $\propto n/\sqrt{p}$ (malla $\sqrt{p} \times \sqrt{p}$, Unidad 4.3) con modelo $t_s + t_w m$ (Unidad 2); $T_{par} = T \times \text{suma}$
4.8	p_{opt} , escalabilidad débil $\leftarrow p_{opt} = \text{mínimo de } T_{par}(n, p)$ (mismo método que el ejemplo KNN, Unidades 4.2/4.3); $n = \Theta(\sqrt{p})$ de $E(n, p)$ constante

Cuadro 4: Mapa de derivación de la Sección 4: cada fórmula nueva se obtiene de la anterior por conteo directo, un resultado citado, o una multiplicación/sustitución algebraica.

4.9. Resumen de complejidades

La Tabla 5 consolida los resultados de esta sección y los enlaza con las tres métricas que pide el punto 1 de la rúbrica (tiempo, *speedup*, eficiencia).

Magnitud	PRAM (sin comunicación)	Memoria distribuida (MPI real)
Pasos de tiempo T	$\Theta(n^2)$	$\Theta(n^2)$
Trabajo / T_1	$\Theta(n^4) = \Theta(N^2)$	$\Theta(n^4)$
Profundidad D_∞	$\Theta(n^2 \log n)$	–
Tiempo T_p	$\Theta\left(\frac{n^4}{p} + n^2 \log p\right)$	$\Theta\left(\frac{n^4}{p} + \frac{n^3}{\sqrt{p}} + n^2 t_s \log p\right)$
<i>Speedup</i> S_p	$\frac{1}{1 + p \log p/n^2}$	$\frac{1}{1 + \Theta(\sqrt{p}/n) + \dots}$
Eficiencia E_p	$\frac{1}{1 + p \log p/n^2}$	$\frac{1}{1 + \Theta(t_w\sqrt{p}/n) + \Theta(t_s p \log p/n^2)}$
Procesos útiles	$O(n^2/\log n)$	$p_{opt} = \Theta(n^2)$ (constante chica)
Escalabilidad débil ($E \approx \text{const.}$)	$n^2 = \Theta(p \log p)$	$n = \Theta(\sqrt{p})$

Cuadro 5: Resumen de complejidades teóricas: modelo PRAM puro vs. extensión a memoria distribuida (MPI).

5. Metodología Experimental y Métricas

Esta sección cubre la parte de **métricas** del punto 1 de la rúbrica y prepara el terreno para los objetivos (b) y (c).

5.1. Definición de las métricas

A partir de los tiempos de pared $T(n, p)$ medidos para cada combinación (tamaño de malla n , número de procesos p), se calculan:

$$S(p) = \frac{T(n, 1)}{T(n, p)} \quad (\text{speedup}), \quad E(p) = \frac{S(p)}{p} \quad (\text{eficiencia}).$$

Estas son exactamente las cantidades S_p y E_p derivadas en las Secciones 4.6 y 4.7, evaluadas con datos reales en vez de la fórmula asintótica.

6. Bibliografía y Uso de IA

6.1. Referencias

Referencias

- [1] W. A. Strauss, *Partial Differential Equations: An Introduction*, 2.^a ed. Wiley, 2008. Referencia base para la ecuación de calor, el estado estacionario y la ecuación de Laplace de la Sección 3.
- [2] R. J. LeVeque, *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*. SIAM, 2007. Cubre la discretización por diferencias finitas (*stencil* de 5 puntos), el esquema FTCS y el análisis de estabilidad de von Neumann usados en la Sección 3.
- [3] R. Courant, K. Friedrichs, and H. Lewy, “Über die partiellen Differenzengleichungen der mathematischen Physik,” *Mathematische Annalen*, vol. 100, no. 1, pp. 32–74, 1928. Origen de la condición de estabilidad CFL aplicada en la Sección 3.3.
- [4] W. Gropp, E. Lusk, and A. Skjellum, *Using MPI: Portable Parallel Programming with the Message-Passing Interface*, 3.^a ed. MIT Press, 2014. Referencia para la descomposición de dominio, el intercambio de *halos* (*ghost cells*) y la comunicación no bloqueante de la Sección 3.5.
- [5] J. JaJa, *An Introduction to Parallel Algorithms*. Addison-Wesley, 1992. Referencia base para el modelo PRAM, las clases EREW/CREW/CRCW y el análisis de trabajo-profundidad (Secciones 4–4.5).
- [6] MIT OpenCourseWare, *18.337J / 6.338J: Parallel Computing and Scientific Machine Learning* – notas de PRAM y modelos de computación paralela. <https://ocw.mit.edu/>. Usado como referencia introductoria al modelo PRAM y su relación con algoritmos numéricos sobre mallas.
- [7] Wikipedia, *Von Neumann stability analysis*. https://en.wikipedia.org/wiki/Von_Neumann_stability_analysis. Base de la condición CFL $\Delta t \leq \min(\Delta x^2, \Delta y^2)/4$ usada en la Sección 3.3 y en la derivación de $T = \Theta(n^2)$ (Sección 4.4).

- [8] A. Grama, A. Gupta, G. Karypis, and V. Kumar, *Introduction to Parallel Computing*, 2.^a ed. Addison-Wesley, 2003. Referencia general para las métricas de *speedup* y eficiencia ($S_p = T_1/T_p$, $E_p = S_p/p$) usadas en la Sección 5.1.
- [9] New Mexico State University HPC, *Serial vs. Parallel Programming with MPI*. <https://hpc.nmsu.edu/discovery/mpi/serial-vs-parallel/>. Usado en la Sección 5 para interpretar por qué el *speedup* y la eficiencia dependen del tamaño del problema n .
- [10] LAMMPS Documentation, *Measuring performance*. https://docs.lammps.org/Speed_measure.html. Define la eficiencia paralela como $\text{rendimiento}(p)/(\text{rendimiento}(1) \times p)$ y la distinción *strong* vs. *weak scaling* usada en la Sección 5.1.

6.2. Uso de Inteligencia Artificial

En el desarrollo de este proyecto y en la redacción de este informe se utilizó asistencia de IA (*Claude*, *Anthropic*) de la siguiente manera:

- **Código (objetivo a/b):** apoyo para depurar el intercambio de halos no bloqueante en `src/heat-mpi-big` redactar los *scripts* de SLURM (`scripts/benchmark.slurm`) y el *script* de recolección de resultados (`scripts/collect_results.sh`).
- **Análisis (objetivo c):** apoyo para estructurar `analysis/plot_performance.py` (cálculo de *speedup*, eficiencia y generación de las cuatro gráficas).
- **Documentación y modelo PRAM (objetivo a):** apoyo para estructurar las derivaciones del modelo PRAM, el análisis trabajo-profundidad, el modelo de costos de comunicación y el análisis de número óptimo de procesos / escalabilidad (`docs/pram-y-complejidad.md`), así como para traducir y reorganizar ese contenido en las Secciones 4–5 de este informe.
- **Redacción de este informe parcial:** apoyo para estructurar el documento L^AT_EX (formato heredado de `Evaluacion_Continua_1.tex`) y redactar las secciones de introducción y metodología.

En todos los casos, **el equipo revisó, validó y es responsable del contenido técnico final:** las derivaciones matemáticas, el código fuente y el contenido presentado en este documento.